Lo básico de cada lenguaje (como se usa en Parkit)

> Archivo personal (no se sube). Solo lo que se usa en el proyecto, con la forma exacta. Comandos y recetas: `COMANDOS-Y-RECETAS.local.md`.

---

1. JavaScript / TypeScript

Escribo	Significa
`const x = 5;`	variable que no cambia (casi siempre esta)
`let x = 5;`	variable que cambia
`` `Hola ${nombre}` ``	texto con variables adentro (comillas invertidas). Permite varias líneas
`a ?? b`	"si `a` es null/undefined, usá `b`": `datos.apodo ?? null`
`a?.b`	"si `a` existe, leé `b`": `usuario()?.id` (no explota si es null)
`a && b` · `a || b` · `!a`	y · o · no
`c ? x : y`	if/else en una línea
`{ ...obj, campo: 1 }`	copia de `obj` con un campo más (o cambiado)
`const { rows } = obj;`	sacar `rows` de un objeto
`const { rows: activas } = obj;`	sacarlo y llamarlo `activas`
`lista.filter((x) => cond)`	quedarse con los que cumplen
`lista.map((x) => otra)`	transformar cada uno
`lista.some((x) => cond)`	¿alguno cumple? (true/false)
`lista.length` · `lista[0]`	cantidad · el primero
`texto.includes('ab')` · `.toUpperCase()` · `.trim()`	contiene · mayúsculas · sin espacios

Funciones

```
// normal
function sumar(a, b) {
  return a + b;
}
// flecha (la más usada)
const sumar = (a, b) => a + b;

// async: cuando adentro hay algo que tarda (ir a la base)
export async function contar(id) {
  const { rows } = await query('SELECT ...', [id]);
  return rows[0];
}
```

- **`await`** espera la respuesta. Sin `await` tenés una promesa vacía.
- Una función con `await` adentro lleva **`async`**.
- **`export`** = otro archivo la puede usar con `import { contar } from '...'`.
- `import * as reservaService from '...'` = todas juntas: `reservaService.contar(...)`.

Tipos (TypeScript)

```
interface Vehiculo {        // la forma de un objeto
  id: string;
  apodo: string | null;     // viene, pero puede ser null
  color?: string;           // puede no venir
}
let ids: string[] = [];     // lista de textos
function x(id: Id): void {} // void = no devuelve nada
```

`interface` solo existe para TypeScript (para avisar errores al compilar); al ejecutar desaparece.

---

2. Angular (front)

La pantalla = clase (.ts) + vista (.html)

```
@Component({
  selector: 'app-vehiculos',
  templateUrl: './vehiculos.html',  // ← su html
  styleUrl: './vehiculos.css',
})
export class Vehiculos {
  private readonly vehiculos = inject(VehiculoService); // pedir un service
  // ...signals, recursos y métodos
}
```

El html usa **todo lo que está en la clase, por nombre** (sin `this.`).

signal — un valor que la pantalla observa

```
protected readonly busqueda = signal('');      // crear (valor inicial)
this.busqueda()                                // leer (con paréntesis)
this.busqueda.set('abc');                      // cambiar
this.contador.update((v) => v + 1);            // cambiar según el anterior
```

Cuando cambia, el html se redibuja solo.

computed — un valor calculado de otros signals

```
protected readonly filtrados = computed(() =>
  this.recurso.value().filter((v) =>
    v.patente.includes(this.busqueda())));
```

Se recalcula solo. No tiene `.set`.

rxResource — cargar datos al abrir la pantalla

```
protected readonly recurso = rxResource({
  params: () => this.idActivo(),              // si cambia, vuelve a pedir
  stream: ({ params }) => this.servicio.listar(params),
  defaultValue: [] as Vehiculo[],             // mientras carga
});
```

Uso	Para
`recurso.value()`	los datos
`recurso.isLoading()`	¿está cargando?
`recurso.error()`	el error, si hubo
`recurso.reload()`	volver a pedir (después de una acción)

Sin `params` (si no depende de nada): solo `stream: () => ...`.

subscribe — para acciones (botones)

```
protected borrar(v: Vehiculo): void {
  this.vehiculos.eliminar(v.id).subscribe({
    next: () => this.recurso.reload(),           // salió bien
    error: (e: Error) => this.error.set(e.message), // salió mal
  });
}
// versión corta (solo el caso bien):
this.servicio.darMeGusta(id).subscribe(() => this.likeado.set(true));
```

El pedido no sale hasta que hacés `.subscribe`. `rxResource` = cargar al abrir · `subscribe` = cuando tocan un botón.

input / output — padre ↔ hijo

```
// HIJO (ej. tarjeta)
readonly reserva = input.required<Reserva>();  // recibe
readonly borrar = output<Reserva>();           // avisa
// en el html del hijo:
<button (click)="borrar.emit(reserva())">Borrar</button>

// PADRE (la pantalla), en su html:
<app-tarjeta [reserva]="r" (borrar)="borrar($event)" />
```

El hijo **no llama al back**: emite. El padre hace el pedido y recarga. `$event` = lo que mandó el `emit`.

Service del front — el único que habla con el back

```
@Injectable({ providedIn: 'root' })
export class EstacionamientoService {
  private readonly http = inject(HttpClient);
  private readonly ruta = '/estacionamientos';  // sin /api

  obtener(id: Id) {                             // GET con mapeo
    return this.http
      .get<{ estacionamiento: EstacionamientoDto }>(`${this.ruta}/${id}`)
      .pipe(map(({ estacionamiento }) => aEstacionamiento(estacionamiento)));
  }
  darMeGusta(id: Id) {                          // POST sin respuesta útil
    return this.http.post(`${this.ruta}/${id}/me-gusta`, {});
  }
  sacarMeGusta(id: Id) {                        // DELETE
    return this.http.delete(`${this.ruta}/${id}/me-gusta`);
  }
  contar(id: Id) {                              // tipo escrito ahí mismo
    return this.http.get<{ filas: { cochera: string; cantidad: number }[] }>(
      `${this.ruta}/${id}/conteo`).pipe(map(({ filas }) => filas));
  }
}
```

Los métodos van **adentro de la clase** (con sangría, al lado de otro).

DTO → mapeo → model

Capa	Archivo	Nombres
DTO (lo que manda el back)	`services/api/api.dto.ts`	`snake_case`
Mapeo (traduce)	`services/api/api.mapeo.ts`	`aX` (llega) · `aPayloadX` (sale)
Model (lo que usan las pantallas)	`models/x.model.ts`	`camelCase`

```
cargadorElectrico: dto.cargador_electrico,    // aEstacionamiento (llega)
cargador_electrico: datos.cargadorElectrico,  // aPayloadEstacionamiento (sale)
```

Formulario reactivo (campo nuevo = 4 lugares)

```
// 1. definición
protected readonly formulario = this.fb.nonNullable.group({
  apodo: ['', [Validators.maxLength(30)]],
  cubierto: false,
});
// 2. precarga al editar (setValue exige TODOS los campos)
this.formulario.setValue({ apodo: v.apodo ?? '', cubierto: v.cubierto });
// 3. al guardar
const valores = this.formulario.getRawValue();
this.guardar.emit({ apodo: valores.apodo.trim() || null });
```

```
<!-- 4. en el html -->
<input formControlName="apodo" />
<input type="checkbox" formControlName="cubierto" />
```

3. HTML de Angular (plantilla)

Escribo	Hace
`{{ x() }}`	muestra un valor
`[value]="x()"`	del .ts al elemento (entra a la pantalla)
`(click)="m()"` · `(input)="..."`	del elemento al .ts (sale)
`(input)="x.set($any($event.target).value)"`	guardar lo que escriben (receta, ver abajo)
`[class.activo]="cond"`	poner una clase si se cumple

```
@if (recurso.isLoading()) {
  <ui-cargando />
} @else if (lista().length === 0) {
  <p>No hay nada</p>
} @else {
  @for (v of lista(); track v.id) {
    <p>{{ v.patente }}</p>
  } @empty {
    <p>Vacío</p>
  }
}

@if (meGusta.value(); as mg) {        <!-- "as": guardarlo con nombre -->
  <span>{{ mg.mio ? '♥' : '♡' }}</span>
}
```

Input conectado a un signal (la receta del `$any`)

```
<input [value]="texto()" (input)="texto.set($any($event.target).value)" />
```

Parte	Qué es
`[value]="texto()"`	lo que se ve en el input sale del signal
`(input)="..."`	se ejecuta en cada tecla
`$event`	el evento (la tecla que tocaron)
`$event.target`	el elemento donde pasó: el `<input>`
`.value`	el texto que tiene escrito
`$any(...)`	"confiá en mí": sin esto TypeScript no deja leer `.value`
`texto.set(...)`	guarda ese texto en el signal

Select → `(change)` en vez de `(input)` · checkbox → `.checked` en vez de `.value` · número → `Number($any($event.target).value)`.

Componentes listos (`components/ui`)

`<ui-boton variante="primario|secundario|fantasma|peligro" (presionado)="m()">` · `<ui-etiqueta tono="aviso|exito|peligro|acento">` · `<ui-cargando />` · `<ui-estado-vacio titulo="..." descripcion="..." />` · `<ui-modal [abierto]="..." (cerrado)="...">`

Clases CSS globales

`campo` (input con estilo) · `label-campo` · `titulo-vista` · `eyebrow` · colores: `text-plomo`, `text-aviso`, `text-acento`, `text-ocupada`.

---

4. Express (back)

La ruta

```
router.patch('/:id/demora', ...soloConductor, controller.avisarDemora);
//     método   URL           portero           a quién llama
router.post('/', ...soloPropietario, validate(validarX), controller.crear);
```

- Porteros: `...soloConductor` / `...soloPropietario` (definidos arriba del archivo como `[authenticate, requireRole(ROLES.X)]`). En `admin.routes.js` ya están para todas (`router.use(...)`).
- `validate(fn)` solo si hay body. Para query: `validate(fn, 'query')`.

El controller (y el req / res)

```
export const avisarDemora = asyncHandler(async (req, res) => {
  const reserva = await reservaService.avisarDemora(
    req.params.id, req.usuario.id);
  res.json({ reserva });
});
```

Dato	De dónde sale	Cuándo
`req.params.id`	la URL (`/:id`)	**a qué cosa**
`req.usuario.id`	el token (lo pone `authenticate`)	**quién pide** (nunca va en la URL)
`req.body`	el body ya validado	datos de un formulario
`req.query`	`?x=1` en la URL	filtros de búsqueda

Respuesta	Cuándo
`res.json({ reserva })`	devolver algo (el nombre es el que lee el front)
`res.json({ filas })`	una lista
`res.json({ ok: true })`	acción sin nada que devolver
`res.status(201).json({ x })`	cuando crea algo

`{ filas }` = `{ filas: filas }` → **tiene que existir la variable `filas`**.

El service y los errores

```
if (!fila) throw ApiError.notFound('No existe');          // 404
if (fila.id_conductor !== id) throw ApiError.forbidden('No es tuya'); // 403
if (fila.llega_tarde) throw ApiError.conflict('Ya avisaste');  // 409
throw ApiError.badRequest('Dato inválido');                // 400
```

409 = está bien pedido pero el estado actual no lo permite. Ante la duda, `conflict`. Un bug sin atrapar = 500 (mirar `[error]` en la terminal).

El validator

```
export function validarX(body) {
  return campos(body)
    .texto('apodo', body.apodo, { requerido: false, max: 30 })
    .entero('anio', body.anio, { requerido: false, min: 1950 })
    .numero('precio', body.precio, { min: 0 })
    .booleano('cubierto', body.cubierto, { requerido: false })
    .enumerado('motivo', body.motivo, ['A', 'B'])
    .uuid('id_vehiculo', body.id_vehiculo)
    .resultado();
}
```

Todo es obligatorio salvo `requerido: false`. **Lo que no está acá se pierde** (llega `undefined`, sin error).

---

5. SQL (Postgres)

Consultar desde el service

```
// sin transacción
const { rows } = await query(
  'SELECT * FROM vehiculo WHERE id_conductor = $1',
  [idConductor],
);
rows[0];      // la primera fila (undefined si no hay)
rows;         // todas

// adentro de withTransaction(async (client) => { ... }) → client.query
```

- **`$1, $2` + array**: el primer elemento llena `$1`. Nunca pegar valores en el texto (`${id}`) → inyección SQL.
- En el texto: nombres de la base (`id_reserva`). En el array: variables de JS (`idReserva`).
- Varias líneas → comillas invertidas `` `...` ``.

Patrones

```
-- leer
SELECT id_vehiculo, patente FROM vehiculo WHERE id_conductor = $1;

-- crear (columnas, $ y array en el mismo orden)
INSERT INTO me_gusta (id_conductor, id_estacionamiento)
VALUES ($1, $2)
ON CONFLICT DO NOTHING            -- si ya existe, no falla
RETURNING *;                      -- devuelve la fila creada

-- cambiar (¡siempre con WHERE!)
UPDATE reserva SET pagada = TRUE WHERE id_reserva = $1;

-- cambiar solo si es suyo (el WHERE es el permiso)
UPDATE vehiculo SET color = $3
 WHERE id_vehiculo = $1 AND id_conductor = $2
RETURNING *;                      -- sin filas → no es suyo → 404

-- borrar
DELETE FROM me_gusta WHERE id_conductor = $1 AND id_estacionamiento = $2;

-- contar / sumar por grupo
SELECT c.identificador AS cochera, COUNT(*)::int AS cantidad
  FROM reserva r
  JOIN cochera c ON c.id_cochera = r.id_cochera
 WHERE c.id_estacionamiento = $1
   AND r.inicio >= date_trunc('month', now())   -- desde el 1 del mes
 GROUP BY c.identificador
 ORDER BY cantidad DESC;

SELECT COALESCE(SUM(precio_total), 0)::float AS total FROM reserva ...;
SELECT ... WHERE estado IN ('PENDIENTE', 'CONFIRMADA');
SELECT ... WHERE nombre ILIKE $1;     -- con '%texto%' en el array
```

Detalle	Por qué
`COUNT(*)::int`	si no, llega como texto `"3"`
`AS nombre`	el nombre con el que llega a JS
`JOIN t ON ...`	unir con otra tabla (con alias: `JOIN cochera c`)
`LEFT JOIN`	igual, pero conserva los que no tienen pareja

Schema (al final de `schema.sql`)

```
-- columna nueva
ALTER TABLE vehiculo ADD COLUMN IF NOT EXISTS apodo VARCHAR(30);
ALTER TABLE estacionamiento
  ADD COLUMN IF NOT EXISTS cargador_electrico BOOLEAN NOT NULL DEFAULT FALSE;

-- tabla nueva que une dos cosas (favoritos, likes, seguir...)
CREATE TABLE IF NOT EXISTS me_gusta (
  id_conductor       UUID NOT NULL
    REFERENCES usuario (id_usuario) ON DELETE CASCADE,
  id_estacionamiento UUID NOT NULL
    REFERENCES estacionamiento (id_estacionamiento) ON DELETE CASCADE,
  PRIMARY KEY (id_conductor, id_estacionamiento)
);
```

- Cada columna: **nombre + tipo + reglas** (`UUID`, `VARCHAR(n)`, `INT`, `BOOLEAN`, `DECIMAL(10,2)`, `TIMESTAMPTZ`).
- Nombres en **minúscula con guion bajo** (Postgres pasa todo a minúscula).
- Sí/no → `NOT NULL DEFAULT FALSE`. Opcional → sin `NOT NULL`.
- Un error acá = el back **no arranca** (`syntax error at or near "X"` → el problema está justo antes de X).

Nombres según dónde

Dónde	Forma	Ejemplo
Base / back / DTO	`snake_case`	`cargador_electrico`
URL	guion medio	`/me-gusta`
JS / TS / model	`camelCase`	`cargadorElectrico`